

BASIC LEVEL (Conceptual & Simple Coding)

1. MCQs

Q1: Which of these is a checked exception?

- a) NullPointerException
- b) IOException ☒
- c) ArithmeticException
- d) ArrayIndexOutOfBoundsException

Q2: Which keyword is used to manually throw an exception?

- a) throws
- b) throw ☒
- c) try
- d) catch

Q3: What will happen if an exception is not caught?

- a) Program terminates ☒
- b) Program continues
- c) Program ignores exception
- d) Compilation error

Q4: Can a try block exist without a catch block?

- a) Yes, only with finally ☒
- b) Yes, always
- c) No
- d) Sometimes

2. Scenario-Based Questions

Q5: Explain what happens if a file is not found when using FileReader.

Answer: FileNotFoundException (checked exception) is thrown. Must be caught or declared with throws.

Q6: What happens if you divide an integer by zero?

Answer: ArithmeticException (unchecked) occurs at runtime, program terminates if uncaught.

3. Coding Questions

Q7: Write a program to handle division by zero.

```
try {  
    int result = 10 / 0;  
} catch(ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

```
}
```

Q8: Write a program to read a file using try-catch.

```
try {  
    FileReader fr = new FileReader("test.txt");  
} catch (IOException e) {  
    System.out.println("File not found");  
}
```

INTERMEDIATE LEVEL (Multiple Catch, Custom Exception, Try-With-Resources)

1. MCQs

Q9: Multi-catch block is supported from which Java version?

- a) Java 5
- b) Java 6
- c) Java 7 ☒
- d) Java 8

Q10: Which exception is thrown when trying to access an invalid array index?

- a) IOException
- b) NullPointerException
- c) ArrayIndexOutOfBoundsException ☒
- d) RuntimeException

Q11: finally block is executed:

- a) Only if exception occurs
- b) Only if no exception occurs
- c) Always ☒
- d) Never

2. Scenario-Based Questions

Q12: How will you ensure a file resource is closed even if an exception occurs?

Answer: Use finally block or try-with-resources.

Q13: How to propagate exception to the caller method?

Answer: Use throws keyword in method declaration.

3. Coding Questions

Q14: Custom exception for age validation.

```
class AgeException extends Exception {  
    AgeException(String msg) { super(msg); }
```

```

}

public class Test {

    static void validate(int age) throws AgeException {

        if(age < 18) throw new AgeException("Age must be >= 18");

    }

    public static void main(String[] args) {

        try { validate(15); }

        catch(AgeException e) { System.out.println(e.getMessage()); }

    }

}

```

Q15: Try-with-resources example.

```

try(BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {

    System.out.println(br.readLine());

} catch(IOException e) {

    e.printStackTrace();

}

```

ADVANCED LEVEL (Exception Chaining, Suppressed, Overriding, Logging)

1. MCQs

Q16: Exception chaining is done using:

- a) getCause()
- b) initCause() ☒
- c) printStackTrace()
- d) finally

Q17: Which of these methods prints exception details to console?

- a) getMessage()
- b) toString()
- c) printStackTrace() ☒
- d) initCause()

Q18: Can a subclass override a method and throw a broader checked exception?

- a) Yes
 - b) No ☒
-

2. Scenario-Based Questions

Q19: How to log exceptions properly in production?

Answer: Use logging frameworks like Log4j, SLF4J. Avoid `printStackTrace()`. Include context info (timestamp, class, method, parameters).

Q20: What happens if multiple exceptions are thrown in try-with-resources?

Answer: The first exception is propagated. Others are suppressed and accessible via `getSuppressed()`.

3. Coding Questions

Q21: Exception chaining example:

```
try {  
    throw new IOException("File error");  
} catch(IOException e) {  
    throw new RuntimeException("Runtime wrapper", e);  
}
```

Q22: Multi-catch example:

```
try {  
    int[] arr = new int[5];  
    arr[10] = 100;  
    String s = null;  
    s.length();  
} catch(ArithmeticException | NullPointerException | ArrayIndexOutOfBoundsException e) {  
    System.out.println("Handled: " + e);  
}
```

Q23: Logging exception using SLF4J:

```
private static final Logger logger = LoggerFactory.getLogger(Test.class);  
  
try { int a = 10/0; }  
  
catch(Exception e) { logger.error("Exception occurred: ", e); }
```